

Deployment of a FortiGate-VM (BYOL) on the AWS

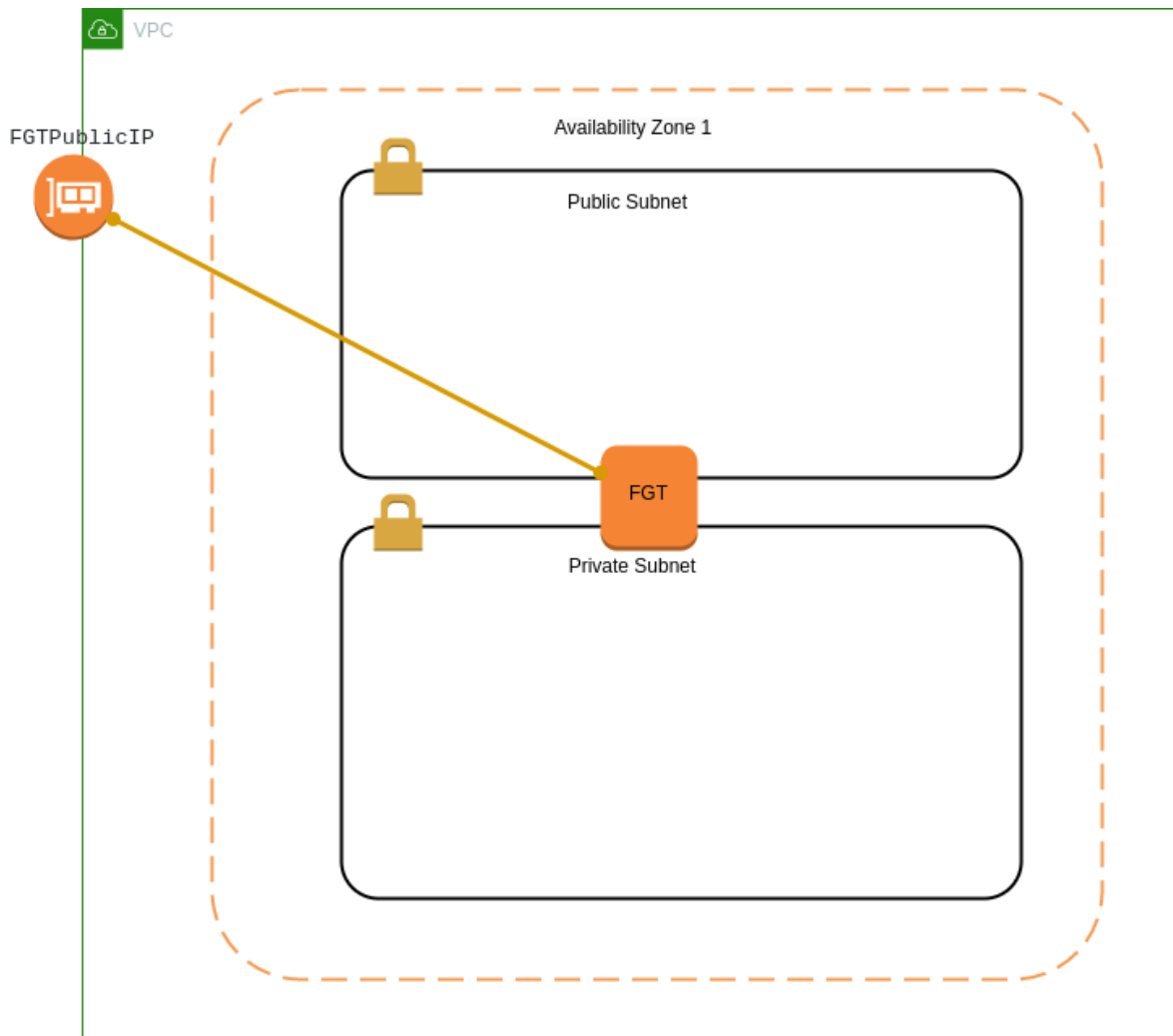
Introduction

A Terraform script to deploy a FortiGate-VM on AWS

Deployment overview

Terraform deploys the following components:

- AWS VPC with 2 subnets
- One FortiGate-VM instance with 2 NICs
- Two Network Security Group rules: one for external, one for internal.
- Two Route tables: one for internal subnet and one for external subnet.



Steps to deploy Single FGTVM in AWS

Step - 1: Update the details

Update the required details in variable.tfvars

Step - 2: Upload a required file

You need store the license file and config file locally and Terraform uploads them automatically.

Upload a file provider.tf, main.tf, variable.tf and variable.tfvars

Step – 3: Run s3_bucket.tf

- ✓ creates an S3 bucket
- ✓ Stores FortiGate **BYOL license** in S3
- ✓ Stores FortiGate **bootstrap configuration** in S3
- ✓ You do NOT need to upload anything to S3 manually.
- ✓ Supports **automated, zero-touch FortiGate deployment**
- ✓ Commonly used with **S3 VPC Endpoint (no internet needed)**

Step – 4: Run network.tf

1 VPC creation

- ✓ Creates a dedicated **VPC** for the FortiGate VM
- ✓ Enables DNS (required for AWS services, S3, etc.)
- ✓ Uses a CIDR block provided via variables
- ➔ This is the **network boundary** for all FortiGate resources.

2 Public and Private subnets

- ✓ Public subnet:
 - Used for **FortiGate external interface (port1)**
 - Connects to the internet

✓ Private subnet:

- Used for **internal interface (port2)**
- Hosts protected workloads

➔ This matches a **classic firewall design**: Outside & Inside.

3 S3 VPC Endpoint

✓ Created only if `var.bucket = true`

✓ Allows **private access to S3** from inside the VPC

✓ No internet or NAT required

✓ Used for:

- FortiGate license download
- Bootstrap config download

➔ Improves **security and automation**.

4 Internet Gateway

✓ Enables internet connectivity for the VPC

✓ Required for:

- FortiGate public interface
- Management access
- Updates

5 Route tables (Public & Private)

✓ Public route table:

- Sends **0.0.0.0/0** → **Internet Gateway**
- Used by public subnet

✓ Private route table:

- Sends **0.0.0.0/0** → **FortiGate internal interface**
- Ensures all outbound traffic flows through firewall

➔ This makes FortiGate the **default gateway** for private traffic.

6 Route table associations

✓ Links:

- Public subnet → public route table
- Private subnet → private route table

→ Without this, routing won't work.

7 Network interfaces (FortiGate ports)

✓ eth0 (port1):

- Public subnet
- Internet-facing

✓ eth1 (port2):

- Private subnet
- Source/destination check disabled (required for firewalls)

→ These map directly to **FortiGate port1 and port2**.

8 Security group attachments

✓ Attaches:

- Public security group → public interface
- Private security group → private interface

→ Controls traffic allowed **to and through** the firewall.

9 Elastic IP for FortiGate

✓ Allocates a **static public IP**

✓ Attaches it to FortiGate public interface

→ Used for:

- Admin GUI
- SSH access
- Stable external endpoint

Step – 5: Run Security-group.tf

This Terraform script creates **two permissive security groups**—one for the public interface and one for the private interface of FortiGate—so that **all traffic flows through the FortiGate firewall, which performs the actual security enforcement.**

This code creates **two AWS Security Groups**:

1. **Public Security Group** → attached to FortiGate **public interface (port1)**
2. **Private Security Group** → attached to FortiGate **internal interface (port2)**

Both security groups **allow all traffic**, and the **actual traffic control is done by the FortiGate firewall**, not by AWS.

Public Security Group (public-sg)

Purpose

-  Used for FortiGate's **internet-facing interface**

Ingress rule

- Allows **all protocols**
- Allows **all ports**
- From the CIDR defined in public-sg-ingress-cidr

-  Needed so FortiGate can receive:

- Internet traffic
- Admin access (HTTPS/SSH)
- VPN traffic

Egress rule

- ✓ Allows FortiGate to send any outbound traffic

2 Private Security Group (private-sg)

Purpose

- ✓ Used for FortiGate's internal interface, facing private workloads

Ingress & egress

Both rules also allow:

- All ports
- All protocols
- Controlled by CIDR variables

Typical use:

- Ingress from private subnet
- Egress to private subnet

- ✓ Ensures FortiGate can inspect and forward internal traffic freely.

Why “Allow all traffic” is OK here

This is intentional and correct for firewall deployments.

Key principle:

AWS Security Groups provide connectivity, FortiGate provides security.

- Security Groups = coarse network access
- FortiGate = deep inspection, policies, logging

Step – 6 : Run fgvm.tf

This Terraform script **fully automates the deployment of a BYOL FortiGate VM**, including configuration, licensing, SSH access, network interfaces, and secure key handling—following DevOps and firewall best practices.

1 Cloud-init configuration (FortiGate bootstrap + license)

✓ Builds a MIME-formatted cloud-init payload

It has **two parts**:

✓ Part 1: FortiGate bootstrap config

- Uses a **template file**
- Injects variables (e.g. admin HTTPS port)
- Configures the firewall on first boot

✓ Part 2: FortiGate BYOL license

- Supports **token-based** or **file-based** licensing
- Automatically applied during VM startup

➔ Result: FortiGate boots configured and licensed.

2 SSH key generation (automated)

✓ Generates a 4096-bit RSA key pair

✓ Fully automated (no manual key creation)

3 AWS EC2 key pair creation

- ✓ Uploads the public key to AWS
- ✓ Used for SSH / CLI access to FortiGate

4 **Store private key in S3**

- ✓ Uploads the private .pem key to an S3 bucket
- ✓ Stored securely (acl = private)

→ Useful for:

- Recovery
- Automation
- Centralized key storage

5 **FortiGate EC2 instance deployment**

✓ Selects the correct FortiGate AMI based on:

- Region
- Architecture
- License type (BYOL)

✓ Uses generated key pair

✓ Attaches cloud-init user data:

- From S3 (if enabled), OR
- Inline cloud-init (fallback)

✓ Attaches IAM role if S3 is used

✓ Creates system and data disks

✓ Uses a public network interface as primary

6 **Second network interface attachment (internal)**

✓ Attaches private interface as eth1

✓ Required for firewall traffic inspection

→ Matches FortiGate design:

- port1 → public
- port2 → private